COS 568 Assignment 2: Distributed Training of Language Models

NetID: od2961 Liv d'Aliberti

March 20, 2026

1 Setup and Reproducibility

- Task 1 CPU run: 1 node (node701), backend `gloo`-free single-process training.
- Task 1 GPU run: 1 node (node915), single GPU.
- CPU distributed timing runs: Tasks 2(a), 2(b), and 3 on `node702, node703, node704, node705`; 1 rank per node, backend `gloo`.
- CPU Task 4 profiling runs: `node706, node707, node708, node710`; 1 rank per node, backend `gloo`.
- Multi-node GPU distributed runs: 4 nodes, 4 ranks, 1 rank per node, backend `nccl`.
- GPU distributed nodes: `node105, node915, node916, node917`
- Seed: `42`, PyTorch version: `2.9.1+cu128`, commit: `4444ce4`.

2 Task 1: Single-Node Fine-Tuning

Epoch	CPU acc	GPU acc
1	0.5920577617	0.5740072202
2	0.6462093863	0.6389891697
3	0.6534296029	0.6606498195

Table 1: Task 1 evaluation accuracy after each epoch.

Minibatch	CPU loss	GPU loss
1	0.748251	0.778352
2	0.797213	0.785116
3	0.708992	0.688311
4	0.755476	0.738948
5	0.740575	0.708527

Table 2: Task 1 first five minibatch losses.

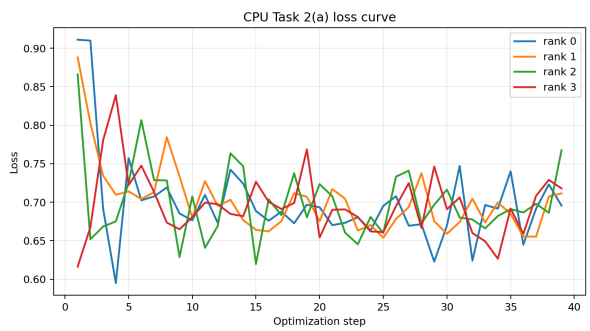
3 Tasks 2(a), 2(b), and 3

Hardware	Task	Method	Avg iter time (s)	Timed iters
CPU	2(a)	gather+scatter	27.086402	38
CPU	2(b)	all_reduce	11.063006	38
CPU	3	DDP	8.161475	38
GPU	2(a)	gather+scatter	2.360095	38
GPU	2(b)	all_reduce	0.754817	38
GPU	3	DDP	0.603755	38

Table 3: Average iteration time after dropping the first iteration.

CPU Task 2(a) vs 2(b) max per-step loss difference: 1.788e-07.
GPU Task 2(a) vs 2(b) max per-step loss difference: 1.788e-07.

CPU Task 2(a)



GPU Task 2(a)

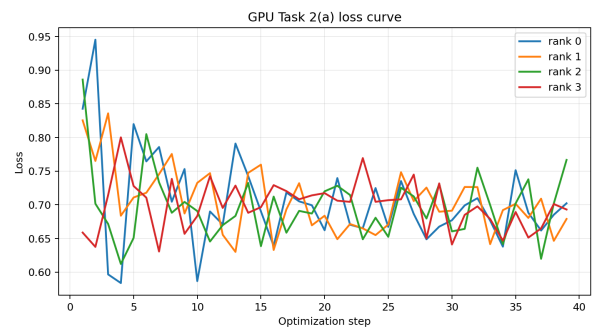
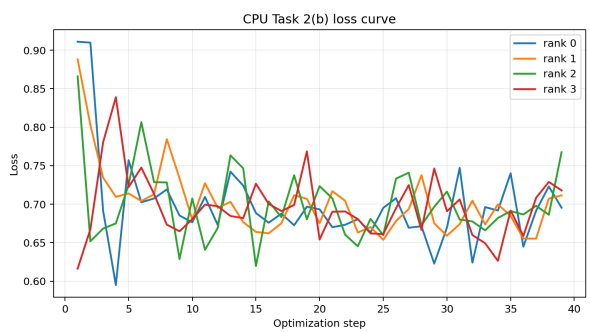


Figure 1: Task 2(a) loss curves across all ranks.

CPU Task 2(b)



GPU Task 2(b)

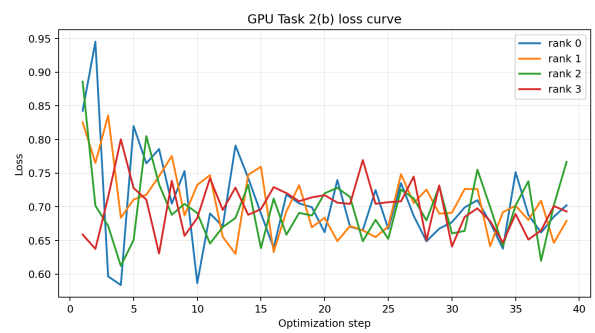


Figure 2: Task 2(b) loss curves across all ranks.

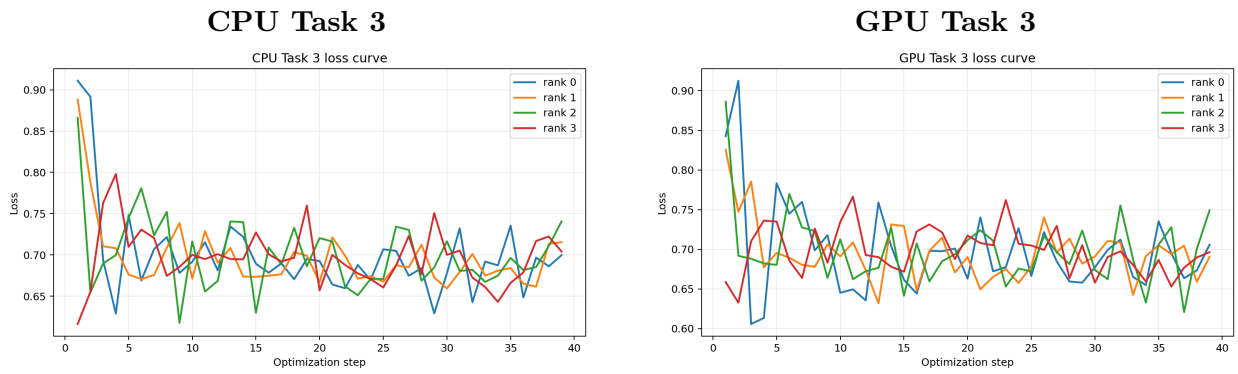


Figure 3: Task 3 loss curves across all ranks.

4 Task 4: Profiling and Communication Overhead

Hardware	Method	Step	Total ms	Comm ms / pct
CPU	gather_scatter	1	28165.536	21336.960 / 75.756%
CPU	gather_scatter	2	27395.801	21310.518 / 77.788%
CPU	gather_scatter	3	27446.430	21308.354 / 77.636%
CPU	all_reduce	1	11357.282	5836.017 / 51.386%
CPU	all_reduce	2	11057.445	5561.979 / 50.301%
CPU	all_reduce	3	11137.599	5557.221 / 49.896%
CPU	ddp	1	8428.010	5639.442 / 66.913%
CPU	ddp	2	8175.506	5449.669 / 66.658%
CPU	ddp	3	8160.515	5271.923 / 64.603%
GPU	gather_scatter	1	2445.820	2139.141 / 87.461%
GPU	gather_scatter	2	2322.969	2177.100 / 93.721%
GPU	gather_scatter	3	2321.385	2177.635 / 93.808%
GPU	all_reduce	1	731.361	598.331 / 81.811%
GPU	all_reduce	2	752.318	623.953 / 82.937%
GPU	all_reduce	3	754.412	625.856 / 82.959%
GPU	ddp	1	621.920	546.223 / 87.828%
GPU	ddp	2	608.071	532.362 / 87.549%
GPU	ddp	3	605.935	531.832 / 87.771%

Table 4: Per-step communication overhead from the profiled three-step windows.

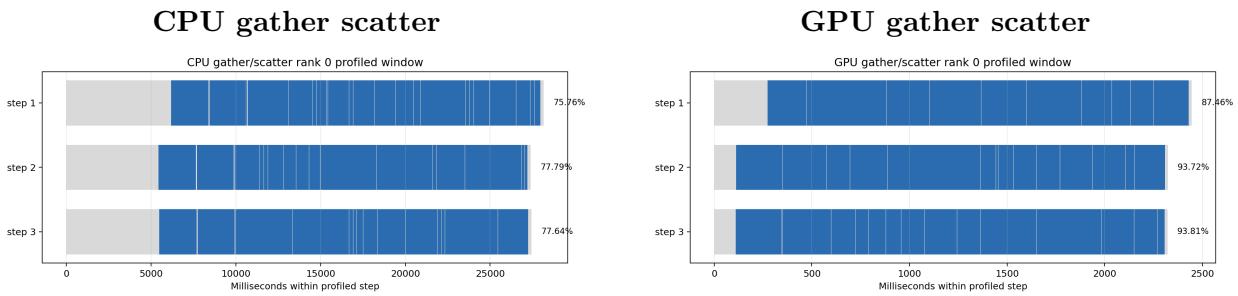


Figure 4: Profiler overview for gather scatter over profiled steps 1-3.

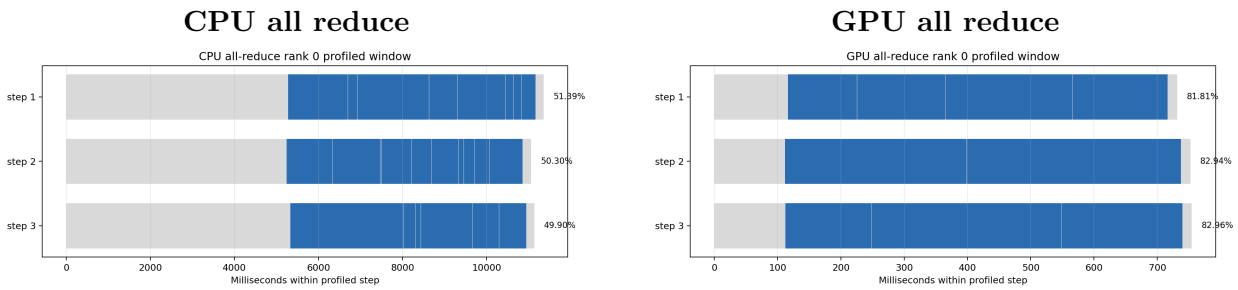


Figure 5: Profiler overview for all reduce over profiled steps 1-3.

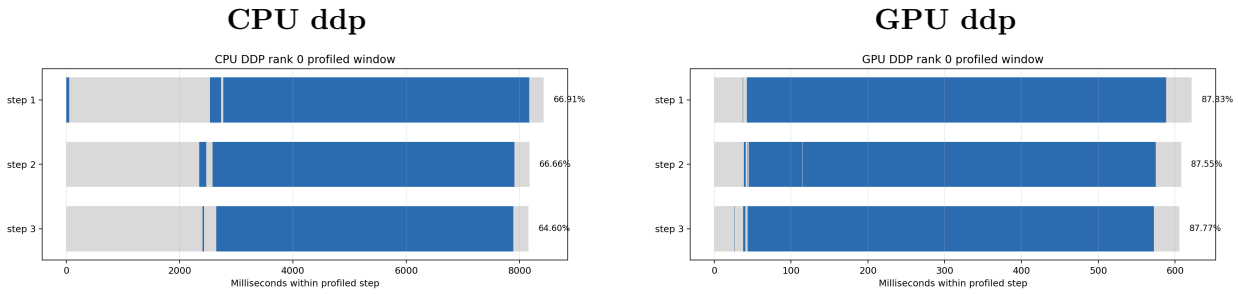


Figure 6: Profiler overview for ddp over profiled steps 1-3.

4.1 Chrome Trace Screenshots

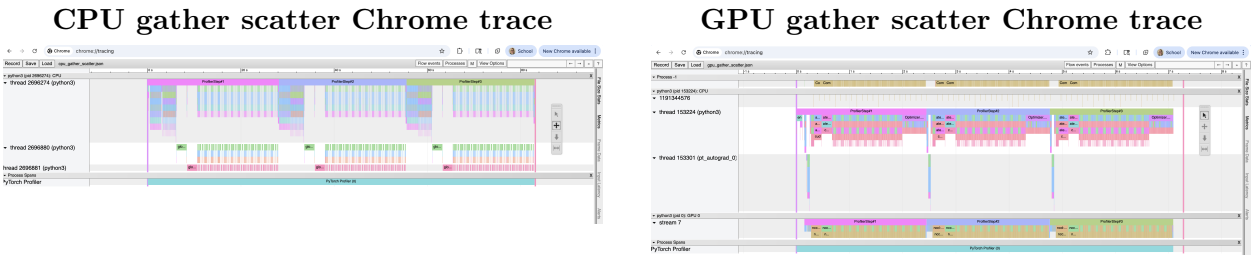


Figure 7: Chrome trace screenshot for gather scatter.

CPU all reduce Chrome trace

GPU all reduce Chrome trace

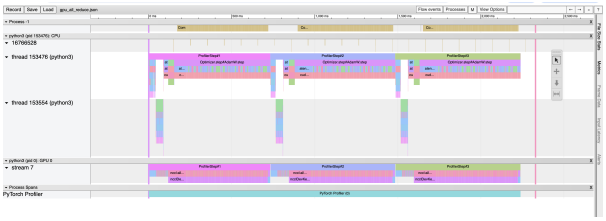
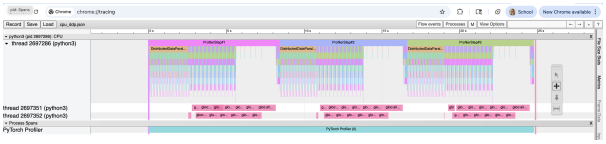


Figure 8: Chrome trace screenshot for all reduce.

CPU ddp Chrome trace



GPU ddp Chrome trace

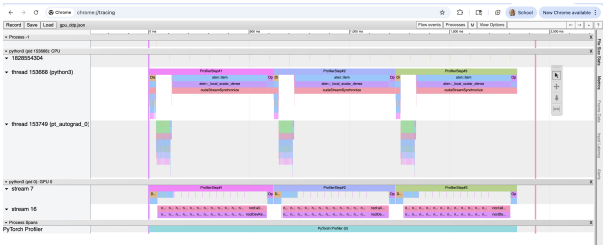


Figure 9: Chrome trace screenshot for ddp.

5 Implementation Details

The code is organized under `task1`, `task2a`, `task2b`, `task3`, and `task4`, matching the deliverable layout requested in the assignment. All distributed runs keep the same total batch size as Task 1 by using per-rank batch size 16 across 4 workers.

Task 1 implements the standard single-node training loop with backward pass, optimizer step, and per-epoch evaluation. Task 2(a) uses a `DistributedSampler` and manual gradient averaging with `gather` followed by `scatter` through rank 0. Task 2(b) replaces that centralized synchronization with `all_reduce` and divides by world size on every rank. Task 3 wraps the model in `DistributedDataParallel`, leaving gradient bucketing and synchronization to PyTorch.

Task 4 reuses the same training code with `torch.profiler` enabled. The profiler schedule skips the first training step and then records the next three steps (`wait=1, warmup=0, active=3`), exporting a rank-0 Chrome trace JSON for each communication method.

6 Discussion

Across both CPU and GPU, the qualitative ordering is consistent: `gather/scatter` is slowest, `all_reduce` is faster, and `DistributedDataParallel` is fastest. On CPU, Task 2(b) is 59.16% faster than Task 2(a), and Task 3 is another 26.23% faster than Task 2(b). On GPU, Task 2(b) is 68.02% faster than Task 2(a), and Task 3 is another 20.01% faster than Task 2(b). The GPU runs are still much faster overall, with GPU/CPU speedups of 11.48x for Task 2(a), 14.66x for Task 2(b), and 13.52x for Task 3.

The Task 2(a) and Task 2(b) loss curves match to within 1.788e-07 on CPU and 1.788e-07 on GPU, which is effectively identical at floating-point precision. That confirms the two manual synchronization implementations are producing the same training trajectory while differing primarily in communication cost.

The lack of difference between Task 2(a) and Task 2(b) in the loss curves is expected. As described in the PyTorch Distributed VLDB'18 paper, distributed data parallel training is designed to be mathematically equivalent to local training when replicas start from the same parameters and synchronize gradients each iteration. In this setup, Tasks 2(a), 2(b), and 3 use the same total batch size and seed, so the main difference among them is communication strategy and runtime cost rather than optimization behavior.

The CPU and GPU runs differ mainly in hardware throughput and backend efficiency rather than in training semantics: GPUs reduce both computation time and collective latency, so the same communication methods preserve ordering but run much faster.

Relative to manual all-reduce, DDP cuts average iteration time by 26.23% on CPU and 20.01% on GPU. That end-to-end reduction is the clearest efficiency comparison for Task 4; the corresponding all-reduce gains over gather/scatter are 59.16% on CPU and 68.02% on GPU.

The Task 4 communication-share metric is computed on rank 0 as the union of all `gloo:/nccl:` intervals inside each `ProfilerStep` window. Under that metric, CPU DDP averages 66.058% versus 50.527% for manual all-reduce (+15.531 percentage points), and GPU DDP averages 87.716% versus 82.569% (+5.147 points). This does not contradict DDP's better end-to-end time: DDP overlaps reduction with backpropagation and uses optimized gradient buckets, so a larger fraction of a shorter step can still be communication-tagged. This is consistent with the PyTorch Distributed design described in the VLDB'18 paper.

These results also illustrate a practical scalability limit for distributed ML. Better collectives clearly improve throughput, but synchronization still occupies a large share of each profiled step, especially once computation becomes faster. In other words, scaling improves when communication is decentralized and overlapped, but communication overhead does not disappear; it becomes the main factor that limits how close distributed training gets to ideal linear speedup.